Simulink Onramp — Complete Notes

Course: Simulink Onramp (`simulinkR2026a` , English) · ~2 hours **Source:** https://matlabacademy.mathworks.com/details/simulink-onramp/simulink **Prepared for:** HACKSIMUL8 2026 — PES University, 2 Sept 2026

> Module and lesson names below are the **actual course structure** read from the live R2026a course page. Technical content is grounded in MathWorks documentation.

Course map (14 modules)

#	Module	Time	Lessons
1	Course Overview	5 min	Course Overview · Running Simulations
2	Simulink Graphical Environment	15 min	Blocks and Parameters · Identifying Blocks and Signals
3	Inspecting Signals	10 min	Inspecting Signals · Simulink Scopes
4	Basic Algorithms	15 min	Mathematical Operators · Basic Logic · Conditional Statements
5	Obtaining Help	5 min	Obtaining Help
6	Project — Automotive Performance Modes	5 min	—
7	Simulink and MATLAB	10 min	MATLAB Workspace Variables · MATLAB Function Block
8	Dynamic Systems in Simulink	5 min	Dynamic Systems
9	Discrete Systems	15 min	Discrete Systems
10	Continuous Systems	10 min	Continuous Systems
11	Simulation Time	5 min	Simulation Time
12	Project — Modeling a Thermostat	10 min	—
13	Project — Peregrine Falcon Dive	10 min	—
14	Conclusion	5 min	Additional Resources · Survey

Module 1 — Course Overview & Running Simulations

What Simulink is. A block-diagram environment for modelling and simulating dynamic systems. You draw the system as blocks connected by signal lines; Simulink numerically integrates it forward in time.

The three things every model has:

1. **Sources** — produce signals (Constant, Step, Ramp, Sine Wave, Signal Editor, From Workspace)
2. **Processing blocks** — transform signals (Gain, Sum, Product, Integrator, Transfer Fcn, Saturation)
3. **Sinks** — consume/observe signals (Scope, Display, To Workspace, Out1)

Running a simulation:

- Set **Stop Time** in the toolstrip (default `10.0`).
- Press **Run** (or `Ctrl+T`, or `sim('modelname')` from the command line).
- Simulink *compiles* first — propagates dimensions, data types and sample times, and checks for algebraic loops — and *then* integrates. Most errors you will meet are compile-time errors: they fire before `t = 0`.

Key mental model: a signal line does not carry a number, it carries a **function of time**. Simulink evaluates the whole diagram at every time step.

Module 2 — Simulink Graphical Environment

Building a model

Action	How
New model	`Ctrl+N` from the Simulink Start Page, or `new_system`
Open Library Browser	`Ctrl+Shift+L`
Add a block by name	**Double-click blank canvas**, type the block name, pick from the list — fastest method by far
Connect blocks	Drag from an output port to an input port
Branch a signal	**Right-drag** (or `Ctrl+drag`) from an existing line
Delete	Select, then `Delete`
Auto-arrange	`Ctrl+Shift+A`

Block parameters

Double-click any block to open its parameter dialog. Parameters can be:

- literal numbers (`5`, `[1 2 3]`)
- **MATLAB workspace variables** (`Kp`) — see Module 7
- MATLAB expressions (`2*pi*f`)

Signals

- **Name a signal** by double-clicking the line. Essential for readability, and for judges.

- Every signal has a **dimension** (scalar / vector / matrix), a **data type** (`double` by default; also `single` , `int8..int64` , `boolean` , `fixdt`), and a **sample time**.
- Mismatches in any of those three are the leading cause of compile errors.

Blocks you will actually use in a hackathon

Block	Library	Purpose
Constant	Sources	fixed value
Step	Sources	step input; set Step time / Initial / Final
Sine Wave	Sources	set Amplitude, Frequency (rad/s), Phase
Gain	Math Operations	multiply by constant `K`
Sum	Math Operations	set **List of signs** to `+-` to form an error signal
Product / Divide	Math Operations	`*` and `/` in List of signs
Integrator	Continuous	`1/s` ; set **Initial condition**
Derivative	Continuous	`du/dt` — avoid, it amplifies noise
Transfer Fcn	Continuous	numerator / denominator coefficient vectors
State-Space	Continuous	A, B, C, D matrices
Saturation	Discontinuities	upper/lower limits — models a real actuator
Unit Delay	Discrete	`1/z` ; breaks algebraic loops
Discrete-Time Integrator	Discrete	forward/backward Euler, trapezoidal
Switch	Signal Routing	pass input 1 or 3 depending on input 2 vs a threshold
Mux / Demux	Signal Routing	bundle / unbundle signals
Scope	Sinks	plot against time
To Workspace	Sinks	export to MATLAB for plotting
Subsystem	Ports & Subsystems	group blocks; keeps the diagram readable

---

Module 3 — Inspecting Signals

Scope

- Double-click to open, run the sim, then **Autoscale** (binoculars icon) to fit.
- **Multiple inputs:** Scope → Settings → Number of input ports, or Mux several signals into one port.
- The **Cursors / Measurements** panel reports rise time, settling time and peak without writing any code — a fast way to quote control performance to judges.

Simulation Data Inspector (SDI)

The modern and better tool.

- Right-click a line → **Log Selected Signals**, run, then open the **Data Inspector**.
- It lets you **overlay runs** — "before tuning" against "after tuning" on one axis. This is the single most persuasive demo artefact in a control hackathon.
- Programmatic: `Simulink.sdi.view`, `Simulink.sdi.createRun`.

To Workspace

Set **Save format** to `Structure With Time` or `Array`, then plot in MATLAB:

```
out = sim('mymodel');
plot(out.simout.Time, out.simout.Data)
```

Module 4 — Basic Algorithms

Mathematical operators

- **Sum** block: the **List of signs** field is what matters. `+-` gives `in1 - in2`; `++-` gives three ports. `|` acts as a spacer for port layout, e.g. `+|-`.
- **Gain**: scalar multiply. For matrix multiply set **Multiplication** to `Matrix(K*u)`.
- **Product**: set **Number of inputs** to `*/` to divide.
- **Math Function**: `exp`, `log`, `sqrt`, `pow`, `mod`, `hypot`.
- **Trigonometric Function**: `sin`, `cos`, `atan2`, and so on.

Basic logic

- **Relational Operator**: `==`, `~=`, `<`, `<=`, `>=`, `>` — outputs **boolean**.
- **Logical Operator**: `AND`, `OR`, `NOT`, `NAND`, `NOR`, `XOR`.
- Logic outputs are `boolean`. Feeding a boolean into arithmetic that expects `double` requires a **Data Type Conversion** block. This is a very common hackathon error.

Conditional statements

Switch is the workhorse:

- Three inputs: `u1` (passed when true), `u2` (control), `u3` (passed when false)
- Criteria: `u2 >= Threshold`, `u2 > Threshold`, or `u2 ~= 0`
- Equivalent to `y = (u2 >= thr) ? u1 : u3`

Also useful: **Multiport Switch** (n-way selection by index), **Saturation** (clip to a range), **Dead Zone**, and **Relay** — which has *hysteresis*, with separate switch-on and switch-off points. Relay is exactly what a thermostat needs; see Module 12.

> **Hackathon tip:** the moment your conditional logic needs *modes*, or memory of which state you were previously in, stop stacking Switch blocks and move to **Stateflow**. See notes file 03.

Module 5 — Obtaining Help

- Select a block and press **F1** for its reference page.
- `doc <blockname>` or `doc <function>` in the Command Window.
- The **Simulink Start Page** carries example models; `openExample('simulink/...')`.
- Right-click a block → **Help**.
- On the day, the local documentation works offline and is faster than web search. Use it.

Module 6 — Project: Automotive Performance Modes

Goal: map an accelerator pedal input to a torque command that behaves differently per mode (Eco / Normal / Sport), using only math, logic and Switch blocks.

Pattern worth remembering:

```
Pedal --> Gain (mode-specific) --+
                                 +--> Switch (mode selector) --> Saturation --> Torque
Pedal --> Lookup / other gain ---+
```

What the project drills: selecting between signals with **Switch**, limiting output with **Saturation**, and composing a mode condition from **Relational** and **Logical** operators.

Module 7 — Simulink and MATLAB

MATLAB workspace variables

Type a variable name (for example `Kp`) into any block parameter field. Simulink resolves it from the base workspace, a model workspace, or a data dictionary, at compile time.

Always do this. Numbers hard-coded across a diagram are neither maintainable nor tunable. Keep a setup script beside the model:

```
% params.m
m  = 1200;    % vehicle mass [kg]
b  = 50;      % damping [N.s/m]
Kp = 800; Ki = 40; Kd = 0;
Ts = 0.01;    % controller sample time [s]
```

Run `params` before `sim`. Better still, set it as the model's **PreLoadFcn** callback (Model Settings → Callbacks) so it runs automatically whenever the model opens.

MATLAB Function block

Write real MATLAB inside a block:

```
function y = fcn(u, k)
%#codegen
    y = k * tanh(u);
end
```

Rules and gotchas:

- Variables must be **fully defined before use** — the block generates code, so growing an array dynamically (`y(end+1) = ...`) fails. Preallocate: `y = zeros(3,1);`
- Sizes and types must be inferable at compile time.
- Not every MATLAB function supports code generation; check the function's doc page.
- For state that persists across time steps, use `persistent`:

```
function y = fcn(u)
    persistent prev
    if isempty(prev), prev = 0; end
    y = 0.9*prev + 0.1*u;
    prev = y;
end
```

- The block is discrete by default, with inherited sample time `-1`.

Use it for algorithms that are ugly as blocks: loops, sorting, matrix operations, custom estimators. **Avoid it for** simple arithmetic, where blocks read better to judges.

Interface summary

Need	Use
Use a MATLAB variable	type its name into any parameter field
Run MATLAB code each step	MATLAB Function block
Get data out to MATLAB	To Workspace / Out1 / signal logging
Get data in from MATLAB	From Workspace / Signal Editor / Constant with a variable

---

Module 8 — Dynamic Systems in Simulink

A **dynamic system** has *memory*: its output depends on past inputs, not only the present one. Mathematically, it has **state**.

- **Static (memoryless):** `y(t) = 2u(t)` — a Gain block.
- **Dynamic:** `y(t)` depends on an integral of `u`, or on `y(t-1)` — it needs an Integrator or a Unit Delay.

The single most important structural fact in Simulink:

> State lives in exactly two kinds of block — **Integrator** (continuous) and **Unit Delay / Discrete-Time Integrator** (discrete). Everything else is a pure function of its current inputs.

That is precisely why algebraic loops occur: a feedback loop built only from memoryless blocks has no state to break the circular dependency.

Direct feedthrough describes a block whose output at time `t` depends on its input at time `t` — Gain, Sum, Product, Switch, Saturation. Integrator and Unit Delay do *not* have direct feedthrough on their main path, which is why they break loops.

Module 9 — Discrete Systems

A discrete system updates only at fixed instants `t = 0, Ts, 2Ts, ...`.

Sample time

Set through the block's `Sample time` parameter. Values mean:

Value	Meaning
`Ts > 0`	discrete, period `Ts`; `[Ts, offset]` also accepted
`-1`	**inherited** — resolved at compile time from the driving block
`0` (or `[0 0]`)	**continuous**
`inf` (or `[inf 0]`)	**constant** — evaluated once at simulation start
`[0 1]`	fixed-in-minor-step — executes only at major time steps
`[-2, Tvo]`	variable sample time; variable-step solvers only

(Values from the MathWorks "Types of Sample Time" reference.)

To see your rates: Debug → Information Overlays → **Sample Time** → *Colors* or *Annotations*. Every distinct rate gets its own colour, which is the fastest way to spot a multirate bug — two blocks you assumed ran together showing up in different colours.

Rate Transition block: required when connecting two different discrete rates. Omitting one gives you either an error or, worse, silently wrong data. Simulink can insert them for you: Model Settings → Solver → *Automatically handle rate transition*.

Core discrete blocks

- **Unit Delay (`1/z`)** — outputs the previous step's input. Has an **Initial condition** parameter. This is your algebraic-loop breaker.
- **Discrete-Time Integrator** — the **Integrator method** parameter matters:
- `Forward Euler`: `y(k) = y(k-1) + Ts*u(k-1)` — no direct feedthrough, so loop-safe
- `Backward Euler`: `y(k) = y(k-1) + Ts*u(k)` — has direct feedthrough
- `Trapezoidal`: `y(k) = y(k-1) + Ts/2*(u(k) + u(k-1))` — most accurate
- **Discrete Transfer Fcn / Discrete Filter** — z-domain coefficients.

- **Memory** — like Unit Delay, for continuous or variable-step contexts.

Turning a difference equation into blocks

`y(k) = a*y(k-1) + b*u(k)` becomes:

```
u --> Gain(b) --> Sum --+--> y
                   ^    |
        Gain(a) <-- Unit Delay <--+
```

Discretising a continuous design

```
Cd = c2d(C, Ts, 'tustin');   % 'zoh' | 'tustin' | 'matched'
```

Rule of thumb: choose `Ts` so the sample rate is **20–40× the closed-loop bandwidth**, i.e. `Ts` between `1/(40*wc)` and `1/(20*wc)`, with `wc` in rad/s.

Module 10 — Continuous Systems

Continuous systems are described by ODEs and integrated numerically by the **solver**.

Building a model from an ODE

For `m*x'' + b*x' + k*x = F`, solve for the highest derivative and integrate downward:

```
F --> Sum --> Gain(1/m) --> [1/s] --> [1/s] --> x
       ^ ^                    x'
       | +--- Gain(b) <--------+     (velocity feedback)
       +----- Gain(k) <---------------------+   (position feedback)
```

The Sum block signs must be `+--`. **A sign error here is the classic cause of a simulation that blows up to `Inf`.**

Integrator block

- **Initial condition** — set it. Physics rarely starts at zero.
- **Limit output** — saturates the state, modelling a tank that cannot overflow or an actuator that cannot exceed its travel. Also gives you anti-windup behaviour.
- **External reset** — rising, falling or level reset of the state.
- **Show state port** — feeds the state to logic without creating an algebraic loop.

Transfer Fcn block

`num` and `den` are descending-power coefficient vectors. `(s+2)/(s^2+3s+5)` means numerator `[1 2]`, denominator `[1 3 5]`. Note that the Transfer Fcn block's initial conditions are always zero — use State-Space when you need non-zero ICs.

Solvers

Solver	Type	Use for
`ode45`	variable-step, explicit	**default**; non-stiff systems
`ode23`	variable-step, explicit	non-stiff, crude tolerance
`ode15s`	variable-step, implicit	**stiff** systems
`ode23t`	variable-step	DAE / moderately stiff, no numerical damping
`ode23tb`	variable-step	stiff; recommended for Simscape Electrical
`ode3`	fixed-step, explicit	non-stiff, fixed rate
`ode14x`	fixed-step, implicit	stiff / DAE at a fixed rate
discrete	fixed or variable	models with **no continuous states**

(Table from the MathWorks "Types of Solvers" reference.)

Choosing one:

- Leave it on `auto` first — Simulink inspects the model's stiffness and state types and picks for you.
- **Simulation crawls, taking tiny steps** → the system is **stiff** → move to `ode15s`.
- **Response looks jagged or misses events** → reduce **Max step size** in the Solver pane.
- **Real-time, code generation, or a fixed rate** → use a fixed-step solver.
- **Frequent switching** (relays, saturation, Stateflow) → fixed-step often behaves better than variable-step, which keeps resetting at zero crossings.
- **Zero-crossing detection** is what lets a variable-step solver land exactly on a discontinuity. If a simulation hangs reporting "excessive zero crossings", either disable ZC on the offending block or switch to a fixed-step solver.

Module 11 — Simulation Time

- **Stop time** sits in the toolstrip; `inf` runs until you press Stop.
- Simulation time is **not** wall-clock time. A 10-second simulation may take 0.2 s or five minutes.
- Programmatic control:

```
out = sim('mymodel', 'StopTime', '20');
set_param('mymodel', 'StopTime', '20');
```

- **Choose the stop time from the physics** — at least 5–10 time constants, long enough to show the response settling. A control demo that stops before settling looks broken.
- The **Clock** block gives current simulation time as a signal, useful for time-varying references. Use **Digital Clock** in discrete models.

Module 12 — Project: Modeling a Thermostat

Concept: a discrete dynamic system with **hysteresis**.

- Room temperature behaves as a first-order lag toward ambient, driven by heater input.
- The controller is not proportional; it is bang-bang with a dead band. Heat goes **on** below `Tset - delta` and **off** above `Tset + delta`.
- Implement with the **Relay** block (Discontinuities library):
- *Switch on point* = `Tset + delta`, *Switch off point* = `Tset - delta`
- The gap between those two points **is** the hysteresis. Without it, the heater chatters at the solver step rate — a classic bug.
- The plant needs **state**, so use a Unit Delay or a Discrete-Time Integrator.

What the project really teaches: memory and state in discrete systems, and why a memoryless comparator (a bare Relational Operator) cannot do real switching control.

Module 13 — Project: Peregrine Falcon Dive

Concept: a continuous dynamic system — Newton's second law with nonlinear drag.

```
m*dv/dt = m*g - 0.5*rho*Cd*A*v^2
```

- Two Integrators in series: acceleration → velocity → position.
- The drag term is **nonlinear** (`v^2`). Use a Product block or Math Function `square`, and watch the sign so that drag always opposes motion (`-sign(v)*v^2`).
- Set the Integrator **initial conditions** — initial altitude and initial velocity.
- Terminal velocity emerges when drag balances weight, which gives you a sanity check: `v_term = sqrt(2*m*g/(rho*Cd*A))`.

What the project teaches: turning physics into an ODE, integrator chains, initial conditions, and nonlinearity in continuous systems.

Module 14 — Conclusion and next steps

Follow-on courses named by the course itself: **Simscape Onramp**, **Stateflow Onramp**, and **Control Design Onramp with Simulink**.

Fast debugging reference

Symptom	Most likely cause	Fix
"Algebraic loop" error	feedback loop of only direct-feedthrough blocks	insert a **Unit Delay** (discrete) or **Memory**; or use a Forward Euler integrator
Signal reaches `Inf` or `NaN`	wrong sign on a feedback Sum; unstable gain; divide by zero	check the Sum **List of signs** first
Dimension mismatch at compile	a scalar meeting a vector	Mux/Demux, or set the Gain multiplication mode
Data type mismatch	a boolean from logic feeding arithmetic	**Data Type Conversion** block
Runs, but output is flat or zero	Integrator initial condition, or a Constant left at 0	check initial conditions
Simulation extremely slow	stiff system on `ode45`	switch to `ode15s`
Output stair-steps unexpectedly	sample time mismatch	Debug → Information Overlays → Sample Time → Colors
"Excessive zero crossings"	chattering discontinuity	add hysteresis, or move to a fixed-step solver
Runs, no error, wrong answer	initial conditions, units, saturation limits	check units first — they are almost always the problem

Triage order on the day:

1. Is it a **compile-time** error (before `t = 0`) or a **runtime** one?
2. Compile-time → dimensions, data types, algebraic loops, unconnected ports.
3. Runtime divergence → stability, sign of feedback, solver choice.
4. Runs but wrong → initial conditions, units, sample time, saturation limits.